# PyGame Car Crash Tutorial - Part 5

## Contents

PyGame Car Crash Tutorial - Part 5 ..... 1
enemy.py ..... 1
player.py ..... 3
Animation ..... 4
carcrash_5.py ..... 6
Assignment Submission ..... 7

Time required: 30 minutes

It's time to move it, move it.

## enemy.py

We are going to add an update method to the enemy and player classes. The update method will move the sprite at every frame.

We will make one change to the enemy init method, to move the initial position of the enemy car above the program window.

```python
class Enemy(pygame.sprite.Sprite):
    """Define the enemy class and methods"""

    def __init__(self):
        """Construct an enemy object from Sprite class"""

        # Call the constructor of the superclass (pygame.sprite.Sprite)
        super().__init__()

        # Load enemy car image from file into a variable
        self.image = pygame.image.load(
            "./assets/enemy.png").convert_alpha()

        # Get the rectangle area of the player car surface
        self.rect = self.image.get_rect()

        # Get a random location 40 pixels away from the left and the right.
        x = randint(40, config.WIDTH - 40)

        # y is -120, the car starts above the program window
        y = -120

        # Move car to initial position
        self.rect.move_ip((x, y))
```

This method moves the enemy car down the screen each frame. When it reaches the bottom, it goes back to the top to a random horizontal position.

```python
    # -------------------------- UPDATE ----------------------------------#
    def update(self):
        """Update the car's position"""
        # Move the sprite down SPEED pixels at a time
        self.rect.move_ip(0, config.SPEED)

        # When the top of the sprite reaches the botton of the surface
        if (self.rect.top > config.HEIGHT):

            # Get a random location 40 pixels away from the left and right.
            x = randint(40, config.WIDTH - 40)

            # Move car above the program window
            y = -120

            # Move car to beginning position
            self.rect.center = (x, y)
```

## player.py

The player's movement is controlled by the keyboard, left and right. Add the update method to the player's class.

```python
# ---------------------------- UPDATE -------------------------------------#
    def update(self):
        """Update the car's position"""
        # Called each time through the Game Loop
        # Read the keyboard to see if any keys pressed
        pressedKeys = pygame.key.get_pressed()

        # Keep the player on the screen
        # The sprite can't move past the left edge of the surface
        if self.rect.left > 0:

            # Left arrow key pressed, move left 5 pixels at a time
            if pressedKeys[pygame.K_LEFT]:
                self.rect.move_ip(-5, 0)

        # The sprite can't move past the right edge of the surface
        if self.rect.right < config.WIDTH:

            # Right arrow key pressed, move right 5 pixels
            if pressedKeys[pygame.K_RIGHT]:
                self.rect.move_ip(5, 0)
```

We define a **move** method for the Player class that controls the movement of the player. When this function is called, it checks to see if any keys are pressed down or not.

The **if** statements check for 2 keys, **LEFT** and **RIGHT**. If the **if** statement proves true, then the **move_ip()** method is called on **Player.rect** moving it in a certain direction. The **move_ip()** takes two parameters, the first representing the distance to be moved in the X direction and second, the distance to be moved in the Y direction.

There are two if statements, as we are testing for two separate event possibilties.

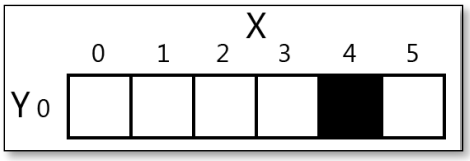
The two if statements, **if self.rect.left > 0:** and **if self.rect.left < config.SCREEN_WIDTH:** ensure that the player isn't able to move off screen.

## Animation

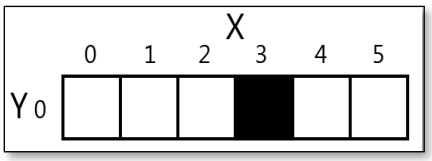
Now that we know how to get the PyGame framework to draw to the screen, let's learn how to make animated pictures. A game with only still, unmoving images is dull. (Sales of the game "Look At This Rock" have been disappointing.)

Animated images are the result of drawing an image on the screen, then a split second later drawing a slightly different image on the screen. Imagine the program's window was 6

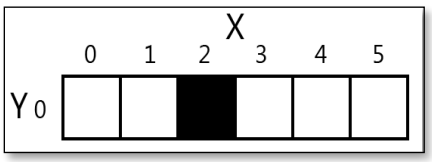
pixels wide and 1 pixel tall, with all the pixels white except for a black pixel at 4, 0. It would look like this:



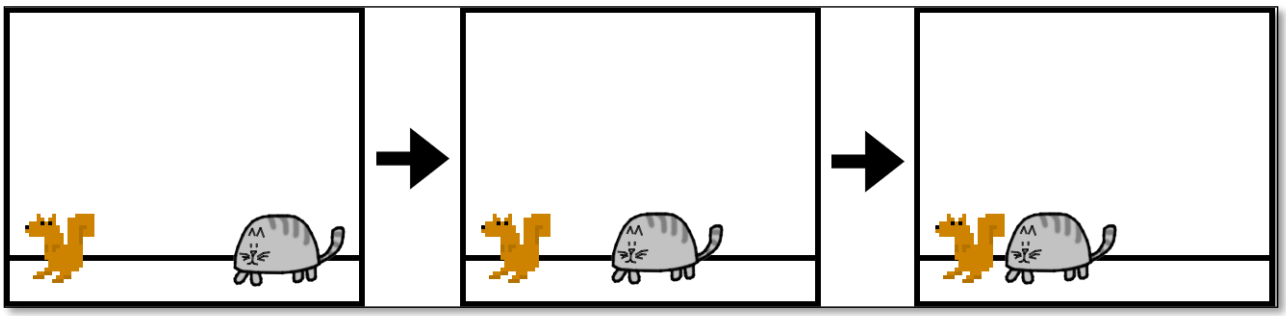

If you changed the image so that 3, 0 was black and 4, 0 was white, it would look like this:




To the user, it looks like the black pixel has "moved" over to the left. If you redrew the window to have the black pixel at 2, 0, it would continue to look like the black pixel is moving left:




It may look like the black pixel is moving, but this is just an illusion. To the computer, it is just showing three different images that each just happen to have one black pixel. Consider if the three following images were rapidly shown on the screen:



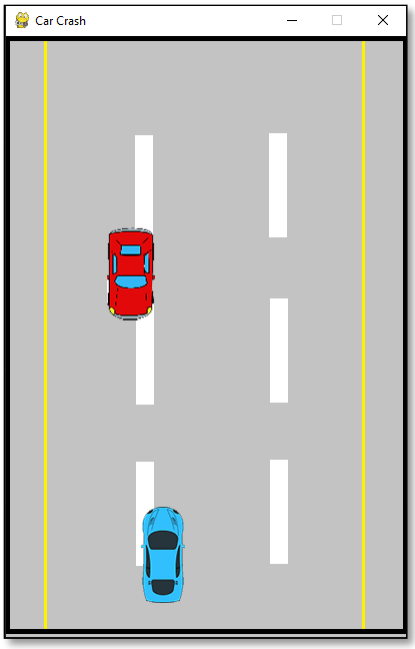
To the user, it would look like the cat is moving towards the squirrel. But to the computer, they're just a bunch of pixels. The trick to making believable looking animation is to have your program draw a picture to the window, wait a fraction of a second, and then draw a slightly different picture.

## carcrash_5.py

One change is needed, call the sprites update method from the main program.

```python
# ------------------------ RUN GAME ----------------------------------------#
    def game_loop(self):
        """Start the infinite Game Loop"""
        while True:
            self.check_events()

            # ------------------ DRAW ON BACKBUFFER -----------------------#
            # Draw everything on the backbuffer first
            # Fill the surface with the background image loaded earlier
            self.surface.blit(self.background, (0, 0))

            # -------------- UPDATE AND DRAW SPRITES ----------------------#
            # Run the update method on all sprites
            self.all_sprites.update()

            # Draw all sprites on the surface
            self.all_sprites.draw(self.surface)

            # ---------------- UPDATE SURFACE ----------------------------#
            # From backbuffer, update Pygame display to reflect any changes
            pygame.display.update()

            # Cap game speed at 60 frames per second
            self.clock.tick(60)
```

Example run:




The player car moves back and forth with the left and right cursor keys.

The enemy car goes down the road nice and smooth at a random location on the X axis.

## Assignment Submission

Zip up the program files folder and submit in Blackboard.